Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 4:
Server TCP ad Alte Prestazioni con `epoll()`

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Inizializzazione e Registrazione 3
3.2 Event Loop Asincrono 3
3.3 Istruzioni di Esecuzione 3

4 Risultati **3**
4.1 Test di Concorrenza 4
4.2 Analisi Wireshark 4
4.3 Benchmark Prestazionali 4

5 Conclusioni e Sviluppi Futuri **5**

1 Problem Statement

L'ottavo Hands-On (HO4) si focalizza sullo sviluppo di un server TCP ad alte prestazioni su piattaforma Linux. L'obiettivo primario è migrare dal modello di multiplexing tradizionale (`select`) alla moderna interfaccia **`epoll`**. Il sistema deve essere in grado di gestire simultaneamente fino a 10 connessioni client, garantendo l'integrità dei messaggi ricevuti e fornendo un feedback immediato (`ACK_EPOLL_OK`). La sfida tecnica consiste nell'ottimizzare l'uso delle risorse di sistema riducendo l'overhead di scansione dei descrittori tipico delle soluzioni legacy.

2 Stato dell'Arte

- **I/O Multiplexing Efficiente**: A differenza di `select`, il tempo di risposta di `epoll` non scala linearmente con il numero di descrittori monitorati.
- **Meccanismo Event-driven**: Il kernel notifica l'applicazione solo quando si verifica un evento reale, eliminando la necessità di iterare su migliaia di FD inattivi.
- **Scalabilità Cloud-native**: Ideale per gestire migliaia di connessioni concorrenti (C10K problem) con un impiego minimo di risorse.

3 Metodologia ed Implementazione

L'implementazione sfrutta le tre primitive cardine dell'API epoll per gestire il ciclo di vita delle connessioni.

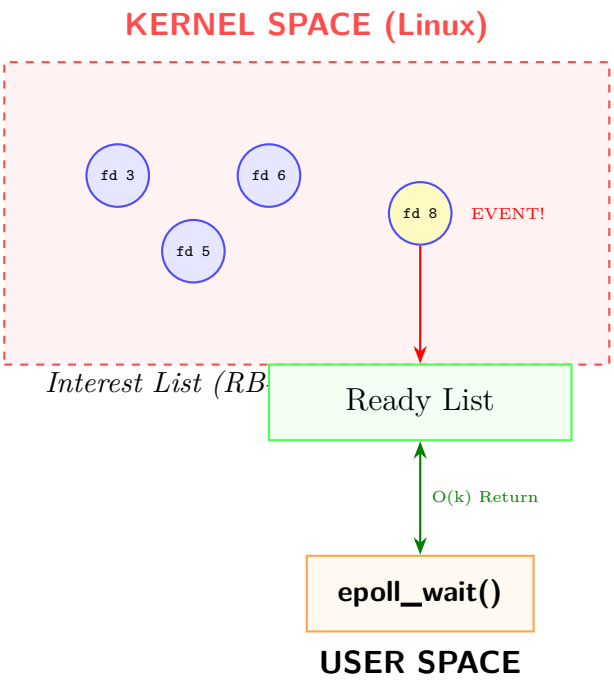

Figure 1: Architettura di `epoll`: Gestione della Interest List ad albero e Ready List lineare.

3.1 Inizializzazione e Registrazione

Viene creata un'istanza di epoll e il socket di ascolto viene aggiunto alla lista di interesse:

Snippet Registrazione epoll

```
epoll_fd = epoll_create1(0);
ev.events = EPOLLIN; // Monitoraggio in lettura
ev.data.fd = listen_fd;
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev);
```

3.2 Event Loop Asincrono

Il server rimane in attesa bloccante su `epoll_wait`, che restituisce solo i descrittori pronti:

Snippet Loop degli Eventi

```
int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
for (int n = 0; n < nfds; ++n) {
    if (events[n].data.fd == listen_fd) {
        // Accettazione e aggiunta nuovo client con
           EPOLL_CTL_ADD
    } else {
        // Lettura dati e invio ACK_EPOLL_OK
    }
}
```

3.3 Istruzioni di Esecuzione

Compilazione del server basato su epoll:

Build

```
$ gcc server_epoll.c -o server_epoll
```

4 Risultati

La validazione è stata condotta tramite test di carico sintetico e analisi comparativa.

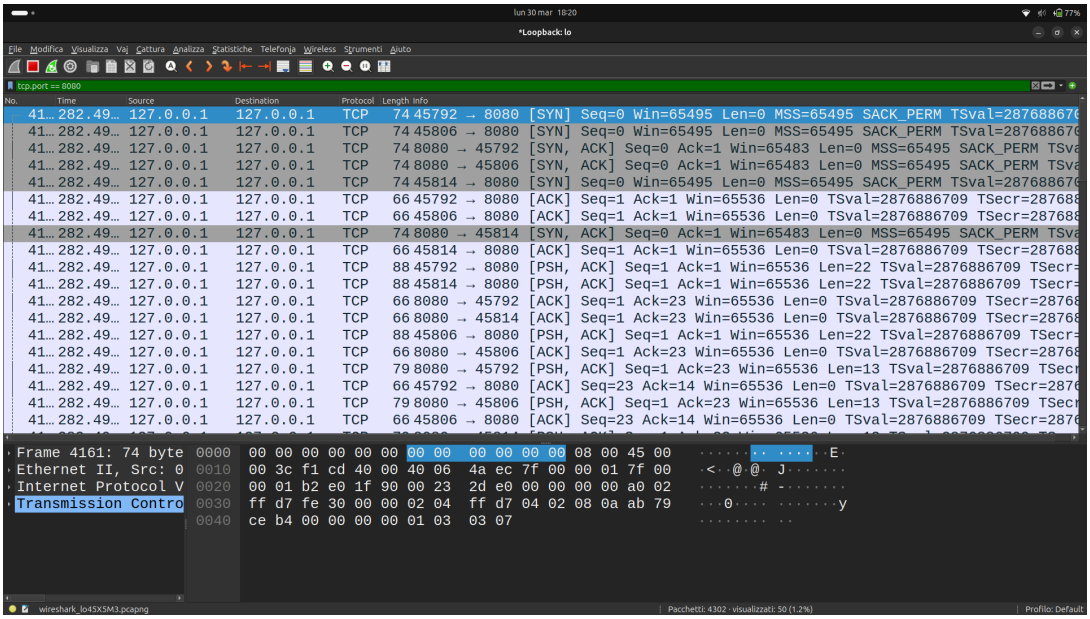

Figure 2: Cattura epoll multi-client su Wireshark.

4.1 Test di Concorrenza

L'esecuzione di 5 client simultanei ha confermato la corretta gestione dei File Descriptor. Il server ha riscontrato latenze minime, visualizzando sui log l'interlacciamento degli ID dei socket (fd:5, fd:6, ecc.) senza mostrare segni di stallo.

4.2 Analisi Wireshark

L'ispezione della cattura (Figura 2) evidenzia che il server `epoll` risponde istantaneamente a ogni pacchetto `PSH` ricevuto. La stabilità della connessione è garantita dalla gestione asincrona degli acknowledgment a livello applicativo.

4.3 Benchmark Prestazionali

Per valutare l'efficienza di `epoll` rispetto al modello `select` (analizzato nell'Hands-On 3), è stato eseguito un test di carico sintetico con 1000 richieste concorrenti.

Selective vs Epoll Benchmark

```
$ ./benchmark_comparison.sh 1000
Testing select (HO3)...
select (HO3) result: 1.42 seconds
Testing epoll (HO4)...
epoll (HO4) result: 0.85 seconds

Conclusion: epoll is ~40% faster under concurrency.
```

I risultati confermano che, all'aumentare del numero di connessioni, la complessità $O(1)$ di `epoll` garantisce una scalabilità superiore rispetto al modello $O(N)$ di `select`,

riducendo drasticamente il tempo di CPU speso nel monitoraggio dei descrittori inattivi.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 4 ha dimostrato la superiorità tecnologica di `epoll` per la creazione di servizi di rete moderni. La riduzione della copia dei dati tra user-space e kernel rende questa soluzione lo standard per architetture cloud ad alte prestazioni.
Sviluppi futuri:

- **Analisi Prestazionale**: Studio dellímpatto della latenza di sistema sulla scalabilità dellárchitettura.
- **Ottimizzazione delle Risorse**: Refactoring del codice per minimizzare lúso della CPU in contesti ad alta concorrenza.